

REPORT DOCUMENTATION PAGE

PLEASE DO NOT RETURN YOUR FORM TO THE ABOVE ORGANIZATION.

1. REPORT DATE 20240614		2. REPORT TYPE Journal Article		3. DATES COVERED	
				START DATE 20200327	END DATE 20240630
4. TITLE AND SUBTITLE Reducing the computational complexity of implicit schemes in the modeling of kinetic inelastic collisions in a partially ionized plasma					
5a. CONTRACT NUMBER		5b. GRANT NUMBER		5c. PROGRAM ELEMENT NUMBER	
5d. PROJECT NUMBER		5e. TASK NUMBER		5f. WORK UNIT NUMBER Q21A	
6. AUTHOR(S) Carl Lederman, David Bilyeu					
7. PERFORMING ORGANIZATION NAME(S) AND ADDRESS(ES) Air Force Research Laboratory (AFMC) AFRL/RQRS 1 ARA DRIVE, EDWARDS AFB, CA 93524-7013				8. PERFORMING ORGANIZATION REPORT NUMBER	
9. SPONSORING/MONITORING AGENCY NAME(S) AND ADDRESS(ES) Air Force Research Laboratory (AFMC) 5 Pollux Drive Edwards AFB, CA 93524-7048				10. SPONSOR/MONITOR'S ACRONYM(S) AFRL/RQR	11. SPONSOR/MONITOR'S REPORT NUMBER(S) AFRL-RQ-ED- JA-2020-085
12. DISTRIBUTION/AVAILABILITY STATEMENT Distribution Statement A: Approved for Public Release; Distribution is Unlimited. PA Clearance Number: 20142; Clearance Date: 02APR20; The U.S. Government is joint author of the work and has the right to use, modify, reproduce, release, perform, display, or disclose the work. Opinions, interpretations, conclusions, and recommendations are those of the author(s) and are not necessarily endorsed by the United States Air Force.					
13. SUPPLEMENTARY NOTES Journal Article published in Numerical Methods for Partial Differential Equations; DOI:doi-org.edwards.idm.oclc.org/10.1002/num.23121 Publication Date (Web):14June2024 ; Copyright ©2024 Wiley Periodicals LLC. ;Prepared in collaboration with ERC Inc. and Parsons Inc. This material is based on work supported by the Air Force Research Laboratory. This is a work of the U.S. Government and is not subject to copyright protection in the United States. Foreign copyrights may apply.					
14. ABSTRACT Modeling the time evolution of atomic number densities and the kinetic (non-Maxwellian) electron energy distribution function under the action of electron impact collisions by classical approaches requires an implicit time-stepping scheme to maintain numerical stability. The resulting linear system that must be iteratively solved at each time step incorporates a dense (nonsparse) matrix. For L variables being propagated, the computational cost is O(L ³).					
15. SUBJECT TERMS Multiscale Modeling, Numerical Partial Differential Equations, Plasma Physics					
16. SECURITY CLASSIFICATION OF:				17. LIMITATION OF ABSTRACT	
a. REPORT Unclassified	b. ABSTRACT Unclassified	c. THIS PAGE Unclassified	SAR		18. NUMBER OF PAGES 25
19a. NAME OF RESPONSIBLE PERSON David Bilyeu				19b. PHONE NUMBER (Include area code) N/A	

RESEARCH ARTICLE

WILEY

Reducing the computational complexity of implicit schemes in the modeling of kinetic inelastic collisions in a partially ionized plasma

Carl Lederman^{1,2}  | David Bilyeu³

¹Astrion (ERC Inc.), Huntsville, Alabama, USA

²Parsons Inc., Chantilly, Virginia, USA

³Air Force Research Laboratory, Edwards AFB, California, USA

Correspondence

Carl Lederman, Astrion (ERC Inc.), 308 Voyager Way NW Suite 200, Huntsville AL 35806, USA and Parsons Inc., 14291 Park Meadow Drive, Suite 100, Chantilly, VA 20151, USA.
Email: carl.lederman@gmail.com

Funding information

Air Force Research Laboratory, Grant/Award Number: 14RQ05COR & 17RQCOR463

Abstract

Modeling the time evolution of atomic number densities and the kinetic (non-Maxwellian) electron energy distribution function under the action of electron impact collisions by classical approaches requires an implicit time-stepping scheme to maintain numerical stability. The resulting linear system that must be iteratively solved at each time step incorporates a dense (nonsparse) matrix. For L variables being propagated, the computational cost is $O(L^3)$. We present an alternative approach with a computational cost of $O(L^2)$, which is the same order as the computational cost of an explicit method for propagating a system of this type. The approach relies on a combination of classical iterative derivative evaluations, combinatorial approximations, and some ideas from deep machine learning.

KEYWORDS

multiscale modeling, numerical partial differential equations, plasma physics

1 | INTRODUCTION

1.1 | Physical model overview

There are various collisional models to choose from, depending on the required level of physical accuracy and available computational resources [5, 35]. On the simpler side of the spectrum is the BGK method [4], which models a simple exponential decay toward a known equilibrium. Elastic collisions of charged particles and atoms can be described by the Boltzmann Equation [5] or the Fokker–Planck Equation [5, 12, 26], which can be derived as an approximation to the Boltzmann equation. These equations are computationally costly, as they involve the velocity distribution function (VDF) of

charged particles, $f(v)$, which provides the number of particles with a given velocity v . The collisionless limit for plasmas is given by the Vlasov equation [8, 37], defined in a domain consisting of 3 spatial and 3 velocity dimensions, and its numerical implementation is computationally expensive. Incorporating collisional models in this equation further increases the computational cost [5]. To reduce computational expenses, various approaches have been explored, such as spatial hybridization [2, 9, 11, 33], spectral expansion [3], implicit spectral methods [10], and delta- f methods [6]. However, most of these models and simplification approaches do not fully incorporate the potentially relevant effects of inelastic collisions, which would add a considerable computational cost to the already expensive problems.

Models of inelastic electron-impact collisions with atoms have been examined in the case of zero spatial dimension and a Maxwellian electron distribution function [1, 19, 38]. Even with these simplifications, the computational cost was high, leading to the development of an approach that groups the raw variables together to produce a simpler system for advancement [1, 19]. Non-Maxwellian electrons and inelastic collisions were modeled in [20], focusing on variable bin sizes for discretizing the electron distribution function and approaches to mass and energy conservation to improve performance. Effectively incorporating collisional-radiative (CR) processes [14, 39] and radiative transport [34] into a model pose considerable challenges.

A comprehensive model incorporating velocity and spatial dimensions while accurately modeling elastic, inelastic, and radiative processes would be massive in scale and likely require several approaches to reduce its computational cost. This research specifically targets one avenue for reducing computational expenses arising from the stiffness of inelastic collisions combined with the large number of variables that need to be propagated.

The model under consideration involves the propagation of a discrete isotropic electron distribution function, f_k , the number density of hydrogen atoms at each excitation level, N_n , and the number density of ions, N_+ . This collection is denoted by u :

$$u(t) \equiv \{f_k(t), N_n(t), N_+(t)\}. \quad (1)$$

The total number of variables being propagated, L , consists of $(K + 1)$ points, with K intervals between these points, at which f_k is discretely evaluated, M atomic excitation levels, and one number for the total number of ions present:

$$L \equiv (K + 1) + M + 1. \quad (2)$$

The propagation in time arises from the sum of the following five physical processes: excitation, de-excitation, ionization, recombination, and elastic electron collisions (Fokker–Planck equation). The map $g : \mathbb{R}^L \rightarrow \mathbb{R}^L$ is defined as:

$$g(u(t)) \equiv \{g_{f_k}(u(t)), g_{N_n}(u(t)), g_{N_+}(u(t))\}. \quad (3)$$

The partial differential equation (PDE) being evolved is then given by [25]:

$$\frac{du}{dt} = g(u) = g_{exc}(u) + g_{de-exc}(u) + g_{ion}(u) + g_{rec}(u) + g_{FP}(u). \quad (4)$$

With discretization of the electron distribution function, Equation (4) becomes an ODE and, with the selection of a time-stepping scheme, a fully discrete system amenable to a computer model that should maintain a high degree of fidelity to the continuous PDE.

While Equation (4) is concisely written, the physical and numerical details encapsulated in each of the g terms are quite complex, and several difficulties must be overcome for the accurate and efficient modeling of this Equation (4). In what follows, when appropriate and for the sake of simplicity, a generic differential equation (DE) is referenced, considered as just:

$$\frac{du}{dt} = g(u). \quad (5)$$

Details of the specific PDE of interest are also amply detailed. Section 2 expands upon each component of the nongeneric Equation (4). Sections 1.2 and 1.3 provide context on the implementation difficulties and the need for new approaches. Sections 3 and 4 present several different approaches to performing the implicit solve needed to advance the solution, and Section 5 provides some computational results.

1.2 | Numerical challenges

For highly excited atoms, the energy gap between levels becomes increasingly small, and the rate of change between them due to electron impact becomes arbitrarily high. This leads to excessive stiffness in the PDE of Equation (4), presenting a challenge that requires specialized schemes [31].

The standard approach to propagating a stiff differential equation is an implicit scheme, and solving the nonlinear equations required to advance the solution with an implicit scheme necessitates repeatedly performing a linear solve with the Jacobian associated with the PDE [31]. For some PDEs, the Jacobian is sparse, allowing for a very fast linear solve. However, the Jacobian associated with Equation (4) is dense and ill-conditioned. Therefore, classical approaches to advancing this PDE require $O(L^3)$ operations, and as discussed in Section 1.1, L can be quite large for this and similar problems.

Some methods attempt to reduce computational cost by incorporating a simplified fluid (nonkinetic, Maxwellian, or near Maxwellian distributions) into the Jacobian calculations [32]. Others approximate a linear solve using Krylov subspaces [18]. A few other methods are presented in Section 3. Some methods for dealing with stiffness attempt to split the problem into slow and fast components [30], but the processes involved in this scenario exhibit a range of rates and do not lend themselves to an obvious splitting. Additionally, other methods aim to reduce computational cost by simply grouping the species number [1, 20] or characterizing the VDF with a reduced basis [36].

1.3 | Motivation

With an eye toward larger systems, we aim to develop numerical schemes that scale as $O(L^2)$ rather than $O(L^3)$, without major simplifications or loss of accuracy. Even an explicit scheme for inelastic collisional PDEs like this requires pairwise evaluations or $O(L^2)$ operations, and therefore stabilizing with a $O(L^2)$ operational cost would not significantly increase the overall expense. Some methods produce stability with a $O(L)$ cost, which is ideal for some situations, but fully taking advantage of the $O(L^2)$ allowance may yield better results. In what follows, we discuss the utility of a few existing methods as well as the advantages of the proposed approach.

2 | COLLISIONAL MODEL

2.1 | Excitation and de-excitation

Atomic electron impact excitation occurs when an electron transfers some of its kinetic energy to excite an atom to a higher energy state. If ε_0 is the electron's kinetic energy before impact, ε_1 is the kinetic energy after impact, and $E_{m \leftarrow n}$ is the energy required to excite an atom from state n to a higher state m . The equation is:

$$\varepsilon_0 = \varepsilon_1 + E_{m \leftarrow n}. \quad (6)$$

When electrons with sufficient energy interact with a number of atoms in state n , some, but not all, of the atoms are expected to be excited. The number of atomic excitations depends not only on the number of electrons and atoms needed for the reaction but also on another physically derived quantity: the excitation cross section, $\sigma_{exc}(v, n)$. The excitation cross section compactly expresses the modulation of the rates at which excitation occurs.

To ensure accurate numerical evaluations, the velocity distribution function (VDF) for numerical modeling purposes is precisely defined as:

$$f(v) = \sum_{k=0}^{k=K} f_k \delta_k(v). \quad (7)$$

Here, $\delta_k(v)$ is a triangle function that takes the value 1 at location kdk and 0 at all other grid locations. While this is a standard finite element implementation with a piece-wise continuous function [16], some challenges specific to this type of modeling problem do present themselves.

For a given number f_{k_0} and another number N_n , we can now provide the precise loss of N_n due to excitation as:

$$g_{N_n} - = N_n \int_v f_{k_0} \delta_{k_0}(v) v \sigma_{exc}(v, n) dv, \quad (8)$$

where $g_{N_n} - = \theta$ is simply defined as the addition of $-\theta$ to g_{N_n} . In this case, $v_0 = k_0 dk$, and v is taken to be in a small area around v_0 . If we were only interested in updating the atomic number and assumed a fixed electron distribution, the entire VDF of Equation (7) could be used in Equation (8), rather than just $f_{k_0} \delta_{k_0}$. However, the current form of Equation (8) allows us to determine exactly where the electrons were taken from as a result of this excitation process. In other words:

$$\int_v g_{exc, k_0} \delta_{k_0}(v) dv - = N_n \int_v f_{k_0} \delta_{k_0}(v) v \sigma_{exc}(v, n) dv. \quad (9)$$

The gain of atoms in the excited state m (where $m > n$) is equal to the loss from state n :

$$g_{N_m} + = N_n \int_v f_{k_0} \delta_{k_0}(v) v \sigma_{exc}(v, n) dv. \quad (10)$$

The gain of electrons, whose energy is set by Equation (6), is given by:

$$\int_v g_{exc, \tilde{k}_1} \delta_{\tilde{k}_1}(v') dv' + = N_n \int_v f_{k_0} \delta_{k_0}(v) v \sigma_{exc}(v, n) dv. \quad (11)$$

Here, $\tilde{k}_1 dk$ represents the location in velocity space where electrons should be added. However, the discrete representation given in Equation (7) only allows for values at specific integer locations. Therefore, two nearby integer locations are selected to place electrons, ensuring mass and energy conservation:

$$\begin{aligned} \int_v g_{exc, \tilde{k}_1} \delta_{\tilde{k}_1}(v') dv' &= \int_{v'} g_{exc, k_{1a}} \delta_{k_{1a}}(v') dv' + \int_{v'} g_{exc, k_{1b}} \delta_{k_{1b}}(v') dv' \\ \frac{1}{2} m_e g \int_{v'} (v')^2 g_{exc, \tilde{k}_1} \delta_{\tilde{k}_1}(v') dv' &= \frac{1}{2} m_e \int_{v'} v'^2 g_{exc, k_{1a}} \delta_{k_{1a}}(v') dv' \\ &\quad + \frac{1}{2} m_e \int_{v'} v'^2 g_{exc, k_{1b}} \delta_{k_{1b}}(v') dv'. \end{aligned} \quad (12)$$

This process in Equation (12) introduces a bit of artificial diffusion, but such a problem is known in the PDE literature and is generally considered acceptable in many cases [27, 31]. For a more detailed consideration of mass and energy conservation issues, we refer to [20].

The equations for de-excitation are similar to those for excitation, with a cross section for de-excitation, $\sigma_{de-exc}(v, n)$, being used instead. In the de-excitation case, electrons involved in the

impact gain energy, and the energy associated with the atom's excitation level decreases as the atom transitions to a lower energy state.

2.2 | Ionization and recombination

A high kinetic energy electron can impact an atom and ionize it, resulting in an unbound ion and two lower energy electrons. Energy is conserved according to the following equation:

$$\varepsilon_0 = \varepsilon_1 + \varepsilon_2 + I_n. \quad (13)$$

For a location k_0 corresponding to an electron with sufficient energy for ionization and a k_1 corresponding to the kinetic energy of one of the post-impact electrons, the number of atoms ionized is given by:

$$g_{N_n-} = N_n \int_v \int_{v'} f_{k_0} \delta_{k_0}(v) v \delta_{k_1}(v') \sigma_{ion}(v, v', n) dv dv'. \quad (14)$$

Within the double integral, v is taken to be near $v_0 = k_0 dk$, while v' (a standard notation for post-collision velocity) is taken to be near $v_1 = k_1 dk$. As with the excitation equations given in Section 2.1, similar expressions to Equation (14) provide the ion gain term as well as the electron loss and gain terms. It is also noted that the calculations are performed with only one type of cross-section for each process type, referred to simply as the cross-section. In other contexts, the σ in Equation (14) may be referred to as a differential cross-section. A more detailed analysis of the physics will often categorize several types of cross-sections, such as the total scattering cross-section, momentum transfer cross-section, and differential cross-section [5].

The rate of ions lost depends on the number of electrons at two locations, along with the number of ions. One of the free electrons becomes bound to the ion to form an ion, while the other is released with extra kinetic energy, satisfying Equation (13). The rate of ion loss is given by:

$$g_{N_+-} = N_+ \int_{v'} \int_{v''} f_{k_1} \delta_{k_1}(v') v' f_{k_2} \delta_{k_2}(v'') v'' \sigma_{rec}(v', v'', n) dv' dv''. \quad (15)$$

Similar equations provide the rate of gain for each atomic species and electron loss and gain. More details on the equations are presented in Appendix A.

2.3 | The Fokker–Planck equation

The Boltzmann Equation has a similar form to the inelastic collisions described in Sections 2.1 and 2.2. However, a 1D Boltzmann equation does not provide the necessary space for a diffusive-like process to occur. Boltzmann collisions tend to result in angular changes that preserve momentum measures. The isotropic distribution could be placed in 2 or 3 dimensions, but this would result in a vast increase in computational cost. Alternatively, the Fokker–Planck equation for isotropic distributions given in [17] works as expected in the modeling of electron-electron elastic collisions. The Fokker–Planck equation is discretized using Chang–Cooper [7] to ensure proper conservation properties and convergence to a Maxwellian distribution over long periods of time.

3 | NUMERICAL METHODS FOR ACHIEVING STABILITY

3.1 | Backward Euler and Newton's method

One simple, first-order accurate, and stable scheme is Backward Euler [31]:

$$u(t + dt) = u(t) + dt \cdot g(u(t + dt)). \quad (16)$$

Here, g , as defined in Equation (3), represents the gain and loss of the components of u , defined by Equation (1), due to the collisional radiative model under consideration. The scheme is implicit, meaning that it requires finding a u that satisfies the following:

$$\begin{aligned} u^* &\equiv u(t), \\ G(u) &= u^* + dtg(u) - u = 0. \end{aligned} \quad (17)$$

To solve Equation (17), the classical approach is to use the Newton–Raphson method [31]:

$$u_{i+1} = u_i - \left(\frac{dG}{du_i} \right)^{-1} G(u_i). \quad (18)$$

Newton’s method requires an initial value for u_0 , which can be set to $u_0 = u(t)$. The Jacobian $\frac{dG}{du_i}$ can be analytically derived or numerically approximated. While numerical approximations are computationally cheaper, they can introduce additional error, potentially requiring more iterations for convergence.

One limitation of Backward Euler and Newton’s method is the computational cost, particularly for problems with a large number of variables. The Jacobian can become dense and ill-conditioned, leading to high computational overhead.

3.2 | Overview of other relevant numerical methods

Other established methods exist that can be applied to the PDE of interest, such as Forward Euler (FE), predictor-corrector (PC), explicit-implicit (IMEX), and Sobolev Regularization (Sobolev). More details on each of these four methods is provided in Appendix C. These other methods are included in the results of Section 5 to provide a broader context for the types of modeling challenges present and the effectiveness of different approaches.

4 | METHODS FOR NONLINEAR REGULARIZATION

4.1 | Combining L^1 and L^2 optimization

As discussed in Section 3.1, the inverse Jacobian is the optimal choice for an operator on G when chosen among all possible linear choices at a particular iteration. However, approximating the linear Jacobian inverse with less computational cost may result in some loss of accuracy. To recover some of the lost accuracy, we explore nonlinear approaches by removing the restriction of the operator being linear at each iteration. One such approach involves using the *ReLU* function from the field of machine learning [21].

The *ReLU* function is defined as follows:

$$ReLU(x) = \max(x, 0). \quad (19)$$

The *ReLU* function evolved from earlier models of neural networks, with each neuron making a binary decision of activation or nonactivation. A positive value is considered a good thing and a path that should be continued down as a part of the learning process. A negative value should be ignored. This is in contrast to a linear process in which a negative value has a negative consequence rather than simply being ignored. This distinction seems subtle, but is the basis of modern deep learning. In most deep learning applications, the returning of a positive or negative value is a very involved process.

For the present purposes, the classification value being worked with is simple. Matter is a positive quantity. A negative value means something went wrong. The learning process that follows cannot incorporate any negative quantity values and does this by use of the *ReLU* function.

For a set of variables, *ReLU* is defined component-wise. It can also be defined for a continuous function $w(x) : \mathbb{R} \rightarrow \mathbb{R}$ as:

$$\text{ReLU}(w(x)) = \max(w(x), 0). \quad (20)$$

Equation (20) can also be interpreted as the composition of an arbitrary function w with the *ReLU* function. This idea is central to deep machine learning [21], where nonlinear functions are constructed from linear functions L_j and *ReLU* using function composition:

$$NL = L_1 \circ \text{ReLU} \circ L_2 \circ \text{ReLU} \cdots. \quad (21)$$

From a regularization perspective, the *ReLU* operator imposes a nonsmooth constraint on a function:

$$w^\dagger = \underset{w}{\operatorname{argmin}} \int (w(x) - f(x))^2 dx \text{ such that } w(x) > 0 \text{ for all } x. \quad (22)$$

This is closely related to the optimization problem given by:

$$w^\dagger = \underset{w}{\operatorname{argmin}} \int |w(x)| dx + \frac{\lambda}{2} \int (w(x) - f(x))^2 dx, \quad (23)$$

where the first term in Equation (23) is an L^1 term and the second is an L^2 term. The known solution to Equation (23) is obtained using the shrinkage function, which provides an approximation to the *ReLU* function when $\lambda \gg f$:

$$w^\dagger = \operatorname{shrink}\left(f, \frac{1}{\lambda}\right) = \max\left\{f - \frac{1}{\lambda}, 0\right\} \approx \text{ReLU}(f) \text{ if } \lambda \gg f. \quad (24)$$

Another possible regularization approach focuses on enforcing that $w^\dagger$ is a strictly increasing function:

$$w^\dagger = \underset{w}{\operatorname{argmin}} \int \left(\frac{dw}{dx} - \frac{du}{dx}\right)^2 \text{ such that } \frac{dw}{dx} > 0 \text{ for all } x. \quad (25)$$

This type of optimization can be efficiently done using split-Bregman [13]. However, it did not seem to offer any significant benefits in computational results compared to simpler approaches using the *ReLU* function and was put aside for the time being.

It is important to note that *ReLU* is incorporated into an operator, specifically the approximation to the Jacobian, rather than directly adjusting u . While adjusting u and re-inserting it into a solver might be a prudent last step, merely adjusting u is unlikely to change the end result significantly. This is similar to changing the initial guess inserted into a Newton solver, which would not affect the end result unless there is a bifurcation or other complicating factors.

4.2 | A 1D operator construction

As an extension of the IMEX method described in Appendix C, we consider a diagonal operator that can incorporate more information from the rapid changes occurring with other variables and the damping being applied there. Simultaneously, the *ReLU* function helps combat insufficient or problematic operator calculations resulting from the nonlinearity of G . A specific example is given later in this section, where a Newton iteration, relying on only a linear characterization of G , inadequately characterizes how the solution should be updated and produces a negative, unphysical value at the next iteration, which causes further problems.

In addition to a primary iteration with iteration number $i \in I$, as in the methods of Section 3, we consider a sub-iteration $j \in J$, where J is small (e.g., 3 as given in the example in Section 5). The notation $u_{j,i}$ simply refers to a u with an iteration number i and sub-iteration number j . The sub-iteration equation is given by:

$$u_{j+1,i} = u_{j,i} - (\tilde{D}NLRd_j, i)^{-1} G(u_{j,i}). \quad (26)$$

An initial guess for $\tilde{D}$ is simply taken from a microscale time step of the DE, with the exact value of the microscale time step chosen small enough to avoid stability issues and large enough to avoid machine precision problems. This sub-iteration is then used to adjust the temporary operator as follows:

$$\tilde{D}_{NLRd}(k, r)_j = \begin{cases} \frac{(G_k(ReLu(u_{j+1,i})) - G_k(u_i))}{ReLu(f_{k,j+1,i}) - f_{k,i}} & \text{if } k = r, \\ 0 & \text{if } k \neq r. \end{cases} \quad (27)$$

In Equation (27), k and r are the matrix indices, and $\tilde{D}$ can be simply applied as a linear operator with matrix-vector multiplication. The nonlinearity arises from the alternate use of $ReLu$ and linear operators as in Equation (21) of Section 4.1. It is also noted that Equation (27) is written in terms of the electron VDF, f , but analogous equations are also implemented for the atomic numbers. After J sub-iterations, the operator to be used in the main iteration is set:

$$D_{NLRd}(k, r)_i \equiv \tilde{D}NLRd(k, r)_J. \quad (28)$$

The update to the main iteration is then:

$$u_{i+1} = u_i - ((D_{NLRd})i)^{-1} G(u_i). \quad (29)$$

A demonstration problem of a 1D cubic ODE that should produce a decay to zero is presented in Figure 1. A first step of Newton's method, with $x(0)$ as the initial guess, produces a negative value at the next time step. An iteration of Newton's method does not incorporate the second or third derivative of G in any way, which explains the possibility of a cubic differential equation potentially producing problematic results. For this demonstration, there are certainly other approaches, such as using an

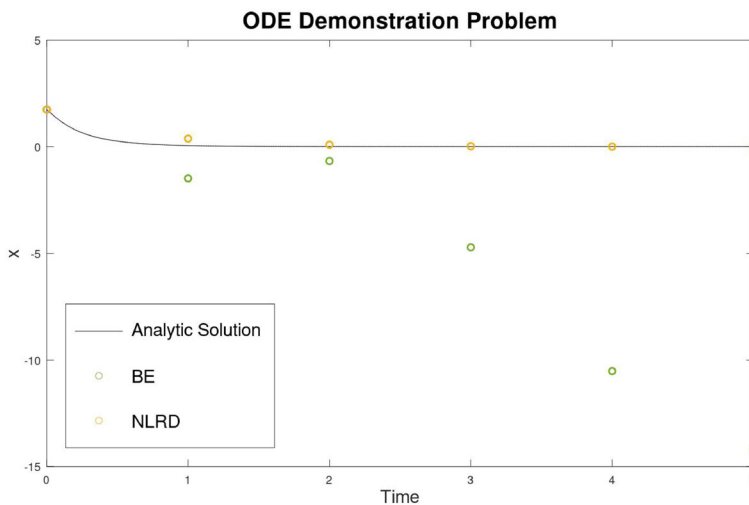


FIGURE 1 The ODE being solved here is $dx/dt = g(x) = x(x+1)(x-3)$, $x(0) = 1.74$, $dt = 1$. On the first time step, Backward Euler jumps to a negative value and does not correct to the long term asymptotic value of 0 on future time steps. The $NLRD$ method (orange) is able to avoid this misstep and produce a qualitatively correct solution.

analog of Backward Euler that incorporates higher order derivatives of g or incorporates a multiple root finding technique. The implementation of Equations (26), (27), (28), and (29), rather than trying to directly address the complexity of nonlinear differential equations, takes a cue from modern deep learning approaches. When led to a value that does not look right, in this case a negative value for what should be a positive quantity, a rejection is made of the decision that led there, the knowledge learned incorporated, and a new better guess attempted.

While a good first step, there are some obvious drawbacks to the construction described in this section. As a solution is approached $u_{i+1} \rightarrow u_i$, the possibility of division by zero under machine precision conditions arises. And even when far from convergence, individual elements of u may be close together. The simple approach taken to address this was to use a Tikhonov type regularization to push an element of $\tilde{DNLrd}(k, k)$ toward 0 in the undefined 0/0 case while, in a second step, forcing $\tilde{DNLrd}(k, k) > \epsilon$ or $\tilde{DNLrd}(k, k) < -\epsilon$ for a small $\epsilon > 0$. While this adjustment mitigated the worst effects of division by 0, it surely still had some negative repercussions. It is noted that other related approaches, such as Krylov solvers, also have a similar ϵ and machine precision issue, and there are a variety of ways that it has been addressed [18]. Another problem with the method described in this section is that it merges the effects of all other variables together. Both of these negative attributes are removed in the method detailed in Section 4.3 below.

4.3 | A 2D operator construction

In this Section, the inverse Jacobian is approximated with a nonlocal mass flow model. Negative mass flow is zeroed out through the use of the *ReLU* function. Each variable is allowed to decay in proportion to itself and allowed to grow from the transfer of mass from elsewhere. The mass lost and gained is reduced using the diagonal components of the Jacobian of g to prevent instability when the rates of decay and growth are high. The off diagonal components of the Jacobian are used to determine to where the mass flows. It is insufficient to only consider the direct paths. For example, more mass may be deposited from considering the transfer $f_{k_3} \rightarrow f_{k_4} \rightarrow f_{k_5}$ than $f_{k_3} \rightarrow f_{k_5}$. Furthermore, it possible that more mass is transferred from the path $f_{k_3} \rightarrow f_{k_4} \rightarrow f_{k_5} \rightarrow f_{k_6} \rightarrow f_{k_5}$ than $f_{k_3} \rightarrow f_{k_5}$. We consider all possible paths, and evaluate the major contributors in the calculation. Considering only direct paths is a fundamentally explicit approach. It is based on what $g(u)$ is indicating at an instantaneous moment in time. The consideration of all paths is fundamentally future looking, taking into consideration the various ways that the system might evolve in time and damping them out.

To start, the two paths consisting of 1 step of length 1 can be defined as:

$$\begin{aligned} P_{[r+1,r]}^1(1) &= 1, \\ P_{[r-1,r]}^1(1) &= -1. \end{aligned} \quad (30)$$

A path with a total number of steps Q , where $q \in Q$, that consists only of steps of length 1, and starts at r and ends at k , is given by:

$$P_{[k,r],s}(q) = \left\{ 1 \text{ or } -1 : \sum_{q=1}^{q=Q} P_{[k,r],s}(q) = r - k \right\}. \quad (31)$$

The integer s in Equation (31) simply denotes one of S possible paths. From the paths, a numerical value is sought that can be aggregated into the operator employed in the implicit solve. The functional φ of a 1-step path from r_1 to r_2 , which can also be referred to as the path value, is given by:

$$\varphi(P_{[r_2, r_1]}^1) = \frac{-ReLu\left(\frac{dg}{du}(r_2, r_1)dt\right)}{-\frac{dg}{du}(r_1, r_1)dt - 1}. \quad (32)$$

The functional φ can then be defined inductively for an arbitrary path, P , through the add-on multiplication of 1-step paths:

$$\varphi(P_{[r_3, r_1]}) = \varphi(P_{[r_3, r_2]})\varphi(P_{[r_2, r_1]}^1). \quad (33)$$

The effect of a location r on the value of location k can then be defined as the sum of the values of all possible paths from r to k :

$$(D_{NLR}^E(k, r)_i)^{-1} = \sum_{s=1}^{s=S} \varphi(P_{[k, r], s}). \quad (34)$$

The letter E indicates that this is an exact, idealized value that is too costly to compute in practice. The number of paths of step number $Q + 1$ and step length 1 is 2 times greater than the number of paths of length Q . To get all paths of length $Q + 1$, one can simply take all paths of length Q and add an additional step up or down to each one. Thus, high values of Q can be neglected in cases where the value of adding an additional 1-step is less than one half or when the flow out tends to be to more than two other locations. As a number of paths with step sizes greater than 1 are also included, the average 1-step path value of length 1 should be even smaller, with around 0.3 being evidently acceptable in practice.

To assist in the approximation of Equation (34), the following path, which starts and ends at the same point, r , and only traverses locations higher than r , is defined:

$$P_{[r, r], s}^+(q) = \left\{ 1 \text{ or } -1 : \left\{ \sum_{q=1}^{q=Q} P_{[r, r], s}^+(q) = 0; \sum_{q=1}^{q=q'} P_{[r, r], s}^+(q) \geq 0 \text{ for all } q' \in Q \right\} \right\}. \quad (35)$$

An analogous type of path for lower locations is given by:

$$P_{[r, r], s}^-(q) = \left\{ 1 \text{ or } -1 : \left\{ \sum_{q=1}^{q=Q} P_{[r, r], s}^-(q) = 0; \sum_{q=1}^{q=q'} P_{[r, r], s}^-(q) \leq 0 \text{ for all } q' \in Q \right\} \right\}. \quad (36)$$

An approximation to Equation (34) is then computed by considering a direct path from $r \rightarrow k$, modified with additional paths that incorporate indirect routes:

$$\begin{aligned} & \text{for } r > k, (D_{NLR}(k, r)_i)^{-1} = \\ & \left(\sum_{s=1}^{s=S} \varphi(P_{[k, k], s}) \right) \varphi(P_{[k, k+1]}^1) \cdots \\ & \varphi(P_{[r-2, r-1]}^1) \left(\sum_{s=1}^{s=S} \varphi(P_{[r-1, r-1], s}^+) \right) \varphi(P_{[r-1, r]}^1) \left(\sum_{s=1}^{s=S} \varphi(P_{[r, r], s}^+) \right). \end{aligned} \quad (37)$$

Equation (37) can then be further expanded to consider different direct paths that include step sizes greater than 1, as well as incorporating larger step sizes onto the indirect add-on paths of Equations (35) and (36). It is also noted that while obtaining a value for $D_{NLR}(k, r)_i$ independently would require a product of $r-k$ elements, the computation can be done as a modification to $D_{NLR}(k, r-1)_i$, preserving a computational cost of $O(L^2)$. We provide more details of the path calculations in Appendix B. By including the most significant paths, we can approximate:

$$(D_{NLR}(k, r)_i)^{-1} \cong (D_{NLR}^E(k, r)_i)^{-1}. \quad (38)$$

The ODE solution at time $t + dt$ is then updated iteratively, similarly to other methods of Section 3:

$$u_{i+1} = u_i - ((D_{NLR})_i)^{-1} G(u_i). \quad (39)$$

The method of this section does not have a sub-iteration of J steps like the one in Section 4.2. However, it can still be seen by inserting the expression for u_{i-1} into Equation (39) and noting that D_{NLR} includes both a linear and *ReLU* component, that the iterative process of Equation (39) still involves an alternating composition of linear and *ReLU* functions.

5 | RESULTS AND DISCUSSION

We applied six different methods presented in Sections 3 and 4 (Backward Euler (BE), Forward Euler (FE), predictor-corrector (PC), explicit-implicit (IMEX), Sobolev Regularization (Sobolev), Non-Linear Diagonal Regularization (NLRD), and Non-Linear Regularization (NLR)) to the ODE of Equation (4) with three different initial conditions. In all seven methods, the initial guess for $u(t + dt)$ was taken to be $u(t)$. For the first two examples, we set the criterion for stopping the iteration to be when the root mean square error achieved by plugging in the approximate solution, $u(t + dt)$, into the Backward Euler Equation (16), was smaller than when the silver standard 'exact' solution was plugged into the same equation. The motivation for this choice was that the schemes are supposed to match a PDE, and there is no real reason to iterate to a level of accuracy that does not match an increased accuracy to the underlying PDE.

The error between arbitrary solutions with A and B labels is defined as:

$$E(u^A, u^B) \equiv \left(\int (f^A(v) - f^B(v))^2 dv + \sum_{n=1}^{n=M} (N_n^A - N_n^B)^2 + (N_+^A - N_+^B)^2 \right)^{1/2}. \quad (40)$$

With SS as the silver standard label, BE as the backward Euler label, and A for an arbitrary label, the stopping criterion is given explicitly as:

$$E(u^A, u^{SS}) < E(u^{BE}, u^{SS}). \quad (41)$$

The criteria given by Equation (41) is carefully chosen to ensure that the iteration number is the minimum number for what can be considered to be a good solution and that extra iterations are not being done that do not produce a meaningful improvement in the solution. The situation was somewhat different for the third example, where the problem was not as stiff as the first two, and a Backward Euler step could converge to a solution with substantial error in comparison to the silver standard 'exact' solution. Therefore, for the third example, error below a fixed low threshold, ε , associated with a qualitatively correct solution, was used:

$$E(u^A, u^{SS}) < \varepsilon. \quad (42)$$

The first example was set with initial conditions corresponding to a partially ionized plasma with excited atomic levels and a Maxwellian electron distribution summed with a narrow Gaussian distribution of electrons. The narrow Gaussian distribution arises from a type of beam with low photon energy. The second example consists of ions, excited atoms, and low-energy electrons. This example was chosen to make the PDE particularly stiff, as higher atomic excitation levels transition very rapidly and the low energy difference between these levels corresponds to a preference for lower energy electrons to make these transitions happen. The third example starts with all atoms in the ground state and

high-energy electrons. The electrons lose energy as they excite atoms from the ground state. The exact formulas for the initial conditions are provided in Appendix D.

The number of iterations required for convergence for example 1 for each of the six methods is given in Table 1. The electron VDF and change in atom number of each species for example 1 are shown in Figures 2 and 3, respectively. The same information for examples 2 and 3 is given in Tables 2 and 3, and Figures 4–7. Figure 8 shows how the solution error changes based on both the iteration number and time step size.

We found that the method that uses an explicit step to iterate, FE, is the worst, as it fails to converge unless the time step is very small. However, the IMEX and Sobolev methods, despite working very differently, are both effective in allowing for stability with time steps at least 10 times larger in comparison to the fully explicit FE approach. The NLR method is generally able to converge with a comparable number of iterations as Backward Euler for small and moderate time steps. But, NLR is able to achieve this convergence with $O(L^2)$ complexity, whereas BE has a computational complexity

TABLE 1 Iteration Numbers for the Different Methods of Example 1.

dt	BE	FE	PC	IMEX	Sobolev	NLRD	NLR
10^{-24}	1	1	2	2	2	1	1
10^{-23}	1	1	4	2	2	2	1
10^{-22}	1	X	X	3	6	3	1
10^{-21}	3	X	X	6	10	8	2
10^{-20}	X	X	X	X	27	X	5
10^{-19}	X	X	X	X	X	X	24
10^{-18}	X	X	X	X	X	X	201

Note: An 'X' indicates a failure to converge. The condition number of the matrix associated with the Backward Euler scheme varied from 1.04×10^4 to 8.48×10^{12} .

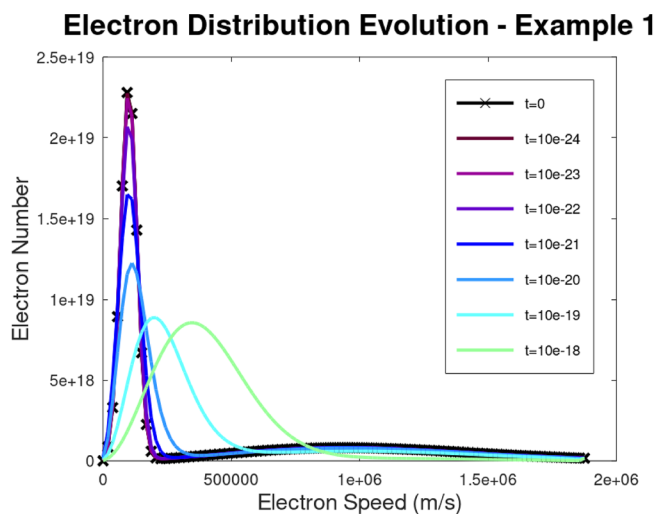


FIGURE 2 Example 1. The Electron VDF from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-24} to 10^{-18} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

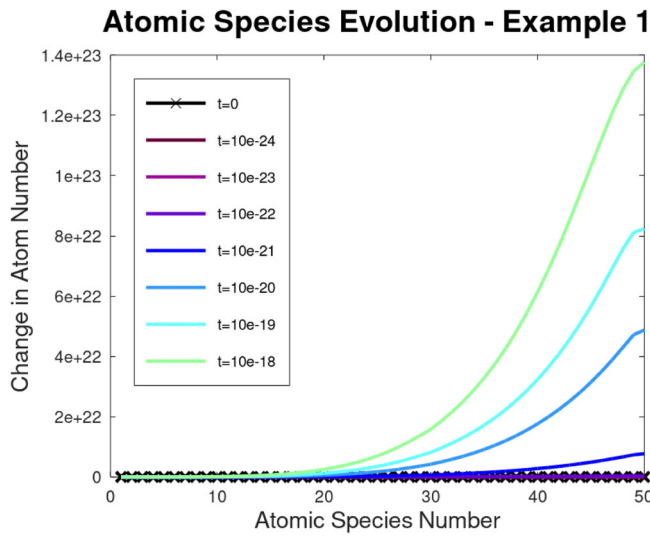


FIGURE 3 Example 1. The change in atomic number for each species type from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-24} to 10^{-18} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

TABLE 2 Iteration Numbers for the Different Methods of Example 2. An 'X' indicates a failure to converge.

dt	BE	FE	PC	IMEX	Sobolev	NLRD	NLR
10^{-24}	1	1	2	2	2	2	2
10^{-23}	2	1	3	2	3	2	1
10^{-22}	X	X	X	10	3	2	3
10^{-21}	6	X	X	X	X	15	4
10^{-20}	8	X	X	X	X	119	6
10^{-19}	X	X	X	X	X	X	13

Note: The condition number of the matrix associated with the Backward Euler scheme varied from 1.16×10^5 to 3.62×10^{13} .

TABLE 3 Iteration Numbers for the Different Methods of Example 3. An 'X' indicates a failure to converge.

dt	BE	FE	PC	IMEX	Sobolev	NLRD	NLR
10^{-15}	1	1	2	2	1	1	1
10^{-14}	1	1	3	3	2	4	2
10^{-13}	2	X	X	11	4	14	4
10^{-12}	2	X	X	182	X	X	12
10^{-11}	4	X	X	X	X	X	34
10^{-10}	5	X	X	X	X	X	94

Note: The condition number of the matrix associated with the Backward Euler scheme varied from 6.65×10^6 to 3.63×10^{17} .

of $O(L^3)$. Figure 9 demonstrates this, with the computation time for one iteration of each approach plotted for various system sizes. For examples 1 and 2, NLR is able to converge for larger time steps than Backward Euler. The stiffness and nonlinearity are considerable at these larger time steps. The ability of NLR to converge for these larger time steps appears to stem from the *ReLU* function, as when it is

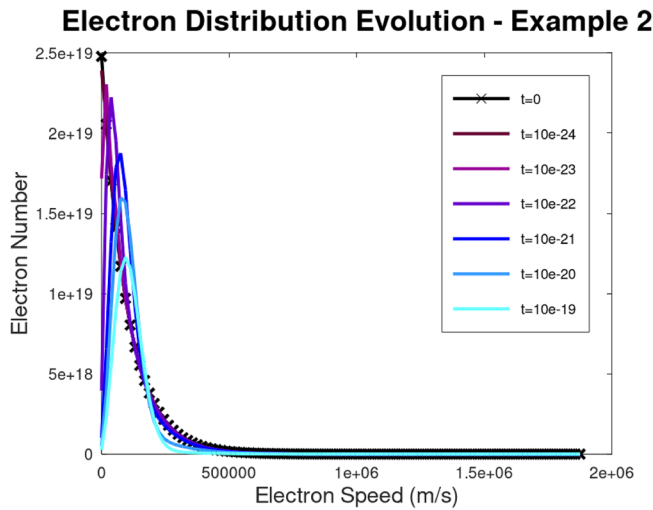


FIGURE 4 Example 2. The Electron VDF from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-24} to 10^{-19} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

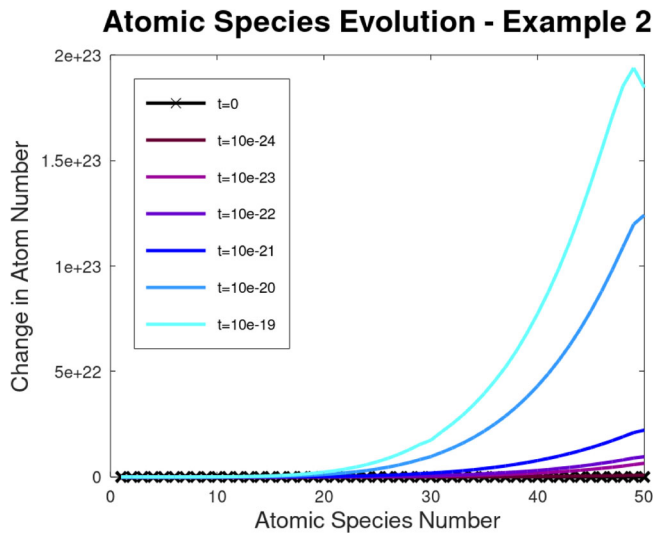


FIGURE 5 Example 2. The change in atomic number for each species type from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-24} to 10^{-19} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

turned off, convergence for this altered NLR does not occur when *BE* does not converge. For example 3, Backward Euler and Newton's method perform better for larger time steps. While both converge, Newton's method does so in fewer iterations. However, we note that the change in the electron VDF for the largest time step size is considerable, transforming almost all the way to a Maxwellian, and this is probably a larger time step size than would be advisable in practice for an accurate solution. With most atoms in the ground state, the stiffness is smaller than in Examples 1 and 2, and the small amount

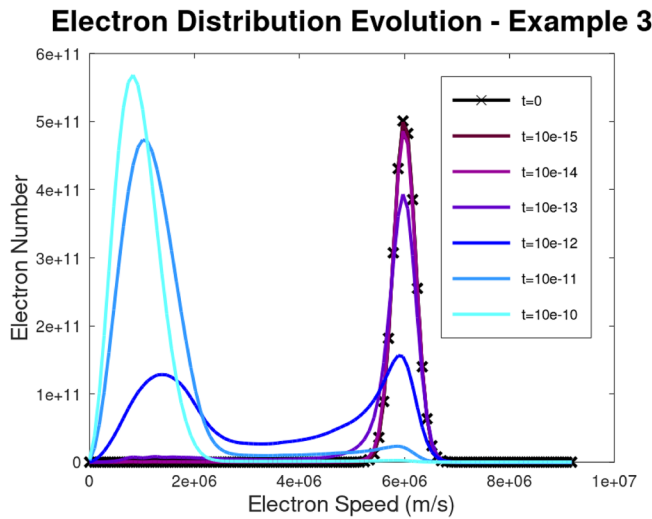


FIGURE 6 Example 3. The Electron VDF from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-15} to 10^{-10} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

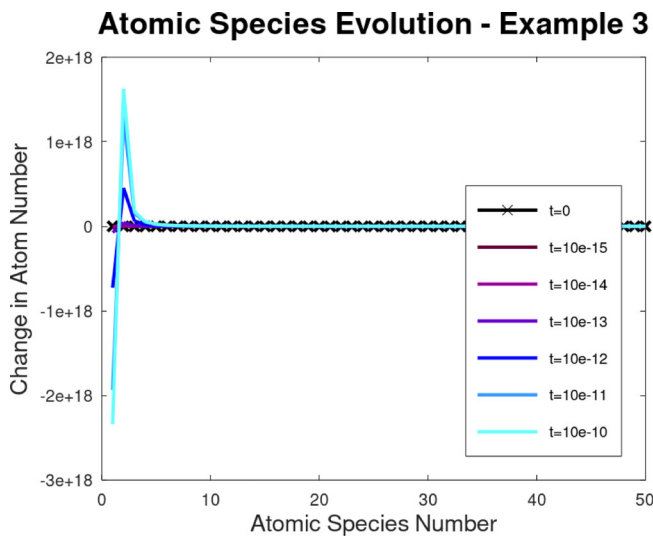
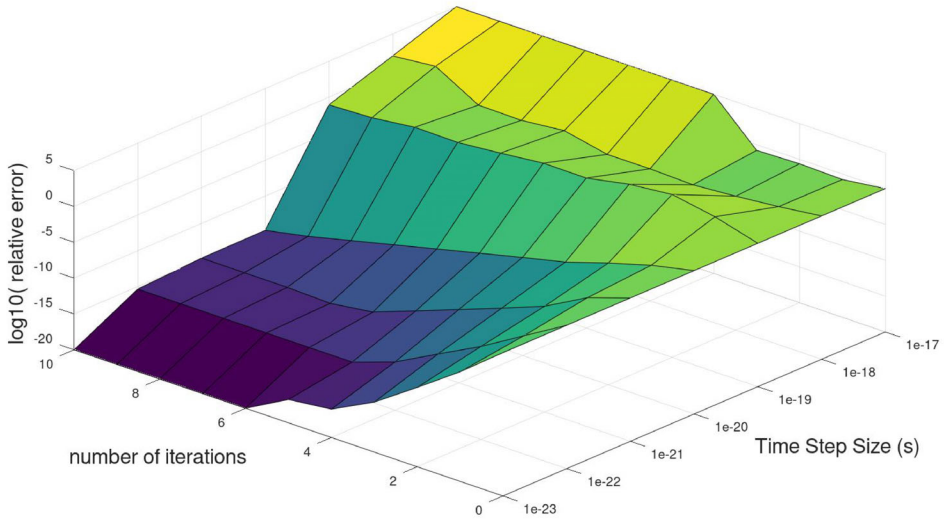


FIGURE 7 Example 3. The change in atomic number for each species type from the PDE of Equation (4) with the Scheme given in Section 4.3 for one time step with sizes ranging from 10^{-15} to 10^{-10} s. The initial condition is drawn in black, and the color transitions from purple to blue to green for larger time values.

of precision lost with the approximations used in the linear component of NLR accounts for the higher iteration count when compared against the precision of Newton's method.

The improvements in both computational cost as well as robustness can only be understood by rejecting the notion that *NLR* is merely an approximation of Backward Euler with Newton–Raphson. Many methods exist that can be fundamentally understood as a coarse approximation of a fine scale solution. The best outcome in this case is that a coarse solution is found when it can be found for

Example 1 relative error with Backward Euler



Example 1 relative error with NLR

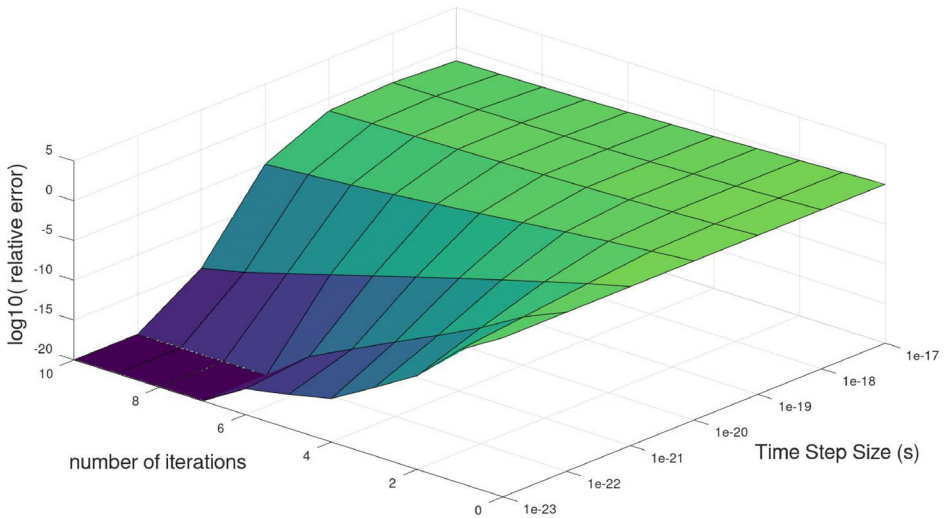


FIGURE 8 The two surface plots show the relative error of two methods as a function of both the number of iterations and the size of the time step. Both methods are able to reduce the error towards machine precision in a few iterations when the time step size is sufficiently small (purple). For larger time step sizes, Backward Euler has a breaking point where volatile growth ensues as the iteration number increases (yellow). NRL still improves at a lower rate of convergence.

the fine solution, the error is small, and that the computational cost is low in comparison to the fine solution. The *NLR* method does not merely approximate the action of repeated linear solves. It offsets the approximation to the linear problem with the incorporation of the nonlinear nature of the PDE. A means of incorporating that nonlinear information reliably and efficiently is an incredible challenge. The approach taken learns from mistakes made, and does so specifically by following the approach of modern deep learning methods with a construction based on alternating between linear operators and *ReLU* functions.

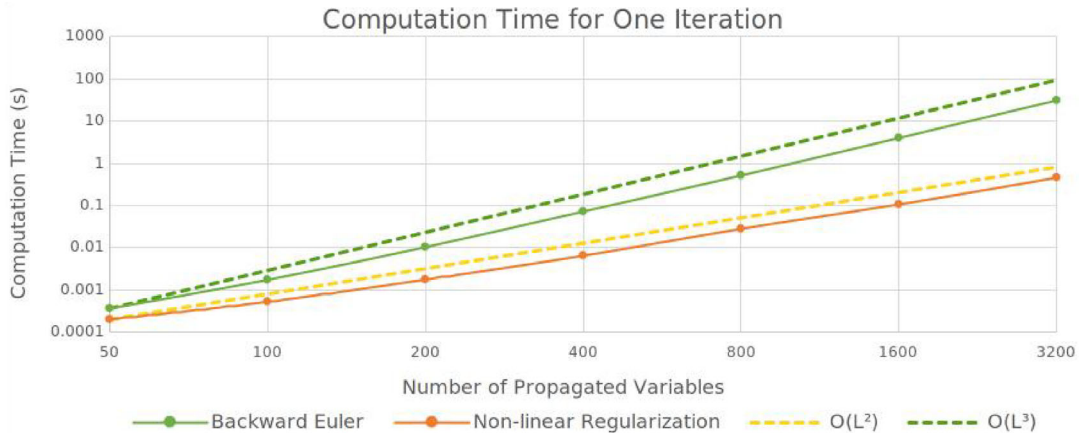


FIGURE 9 Comparison of Computation Times. Computation times for the proposed method are compared against a standard BE scheme. The computational cost for one iteration of BE was dominated by the $O(L^3)$ linear algebra operations using a standard optimized linear algebra package, LAPACK. Tests were performed on a common laptop computer with an Intel(R) Core(TM) i9-8950HK CPU. The dotted lines show an exact squared or cubic increase and are roughly parallel to their corresponding methods as the number of variables being propagated increases. Factors such as the cost associated with setting up the relevant matrices and vectors seem to have some effect when the variable number is small.

6 | CONCLUSION

Accurately modeling a partially ionized plasma is an exceedingly challenging computational task, and ongoing progress in modeling the different types of physics using various computational techniques is essential. In this work, we focused on improving the efficiency of implicit time step advances when modeling inelastic electron-atom collisions with a non-Maxwellian kinetic electron VDF. The potential computational improvement, from a cost of $O(L^3)$ with classical approaches to $O(L^2)$ with the proposed Non-Linear Regularization (NLR) method, is substantial for large L .

The increased efficiency achieved by the NLR method may open up new possibilities for more computationally practical modeling of plasmas with a higher number of excitation and ionization levels, more types of collisional processes, and more dimensions in velocity and space. Moreover, it may complement other approaches aimed at improving computational efficiency, particularly those that manipulate processes in space or velocity space rather than time.

Lastly, the building blocks of the nonlinear operator, particularly the alternating composition of linear and *ReLU* functions, may find applications beyond the realm of deep learning and could be of interest in implicit solves for other types of PDEs and diverse scientific fields. The improvement in computational efficiency achieved by the NLR method presents a promising step towards advancing our understanding and modeling of complex plasma physics.

ACKNOWLEDGMENTS

We would like to thank Hal Cambier for the many helpful insights he provided.

FUNDING INFORMATION

This research was supported by the U.S. Air Force Office Of Scientific Research and U.S. Air Force Research Laboratory Grants 14RQ05COR & 17RQCOR463.

CONFLICT OF INTEREST STATEMENT

No conflict of interest exists. We wish to confirm that there are no known conflicts of interest associated with this publication, and there has been no significant financial support for this work that could have influenced its outcome.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Carl Lederman  <https://orcid.org/0000-0002-0544-7181>

REFERENCES

- [1] R. J. E. Abrantes, A. R. Karagozian, D. Bilyeu, and H. P. Le, *Complexity reduction effects on transient, atomic plasmas*, J. Quant. Spectrosc. Radiat. Transf. 216 (2018), 47–55.
- [2] A. Alaia and G. Puppo, *A hybrid method for hydrodynamic-kinetic flow—part II—coupling of hydrodynamic and kinetic models*, J. Comput. Phys. 231 (2012), no. 16, 5217–5242.
- [3] A. Bar-Shalom, J. Oreg, W. H. Goldstein, D. Shvarts, and A. Zigler, *Super-transition-arrays: A model for the spectral analysis of hot, dense plasma*, Phys. Rev. A 40 (1989), no. 6, 3183.
- [4] P. L. Bhatnagar, E. P. Gross, and M. Krook, *A model for collision processes in gases. I. small amplitude processes in charged and neutral one-component systems*, Phys. Rev. 94 (1954), no. 3, 511.
- [5] J. A. Bittencourt, *Fundamentals of plasma physics*, 3rd ed., Springer, New York, NY, 2003.
- [6] S. Brunner, E. Valeo, and J. A. Krommes, *Collisional delta-f scheme with evolving background for transport time scale simulations*, Phys. Plasmas 6 (1999), no. 12, 4504–4521.
- [7] J. S. Chang and G. Cooper, *A practical difference scheme for Fokker-Planck equations*, J. Comput. Phys. 6 (1970), no. 1, 1–16.
- [8] C.-Z. Cheng and G. Knorr, *The integration of the Vlasov equation in configuration space*, J. Comput. Phys. 22 (1976), no. 3, 330–351.
- [9] P. Degond, G. Dimarco, and L. Mieussens, *A multiscale kinetic–fluid solver with dynamic localization of kinetic effects*, J. Comput. Phys. 229 (2010), no. 13, 4907–4933.
- [10] G. L. Delzanno, *Multi-dimensional, fully-implicit, spectral method for the Vlasov–Maxwell equations with exact conservation laws in discrete form*, J. Comput. Phys. 301 (2015), 338–356.
- [11] G. Dimarco, L. Mieussens, and V. Rispoli, *An asymptotic preserving automatic domain decomposition method for the Vlasov–Poisson–BGK system with applications to plasmas*, J. Comput. Phys. (2014), 122–139.
- [12] A. D. Fokker, *Die mittlere energie rotierender elektrischer dipole im strahlungsfeld*, Ann. Phys. 348 (1914), no. 5, 810–820.
- [13] T. Goldstein and S. Osher, *The split bregman method for l1-regularized problems*, SIAM J. Imaging Sci. 2 (2009), no. 2, 323–343.
- [14] M. Goto, *Collisional-radiative model for neutral helium in plasma revisited*, J. Quant. Spectrosc. Radiat. Transf. 76 (2003), no. 3–4, 331–344.
- [15] R. W. Hamming, *Stable predictor-corrector methods for ordinary differential equations*, J. ACM 6 (1959), no. 1, 37–47.
- [16] C. Johnson, *Numerical solution of partial differential equations by the finite element method*, Courier Corporation, North Chelmsford, Massachusetts, 2012.
- [17] C. F. F. Karney, *Fokker-Planck and quasilinear codes*, Comput. Phys. Rep. 4 (1986), no. 3–4, 183–244.
- [18] D. A. Knoll and D. E. Keyes, *Jacobian-free Newton–Krylov methods: a survey of approaches and applications*, J. Comput. Phys. 193 (2004), no. 2, 357–397.
- [19] H. P. Le, A. R. Karagozian, and J.-L. Cambier, *Complexity reduction of collisional-radiative kinetics for atomic plasma*, Phys. Plasmas 20 (2013), no. 12, 123304.
- [20] H. P. Le and J.-L. Cambier, *Conservative algorithms for non-Maxwellian plasma kinetics*, Phys. Plasmas 24 (2017), no. 12, 122105.
- [21] Y. LeCun, Y. Bengio, and G. Hinton, *Deep learning*, Nature 521 (2015), no. 7553, 436–444.
- [22] C. Lederman, A. Joshi, I. Dinov, J. D. Van Horn, L. Vese, and A. Toga, *A unified variational volume registration method based on automatically learned brain structures*, J. Math. Imaging Vision 55 (2016), no. 2, 179–198.
- [23] D. Mujeeb, J. W. Neuberger, and S. Sial, *Recursive form of Sobolev gradient method for odes on long intervals*, Int. J. Comput. Math. 85 (2008), 1727–1740.

- [24] J. W. Neuberger, *Sobolev gradients and differential equations*, Lecture Notes in Mathematics #1670, Springer, New York, NY, 1997.
- [25] J. Oxenius, *Kinetic theory of particles and photons: theoretical foundations of non-LTE plasma spectroscopy*, Vol 20, Springer Science & Business Media, Heidelberg, Germany, 2012.
- [26] V. M. Planck, *Über einen satz der statistischen dynamik und seine erweiterung in der quantentheorie*, Sitzungsberichte der Preussischen Akademie der Wissenschaften, Germany, 1917.
- [27] K. G. Powell, P. L. Roe, T. J. Linde, T. I. Gombosi, and D. L. De Zeeuw, *A solution-adaptive upwind scheme for ideal magnetohydrodynamics*, J. Comput. Phys. 154 (1999), no. 2, 284–309.
- [28] P. R. Preussner and K. P. Brand, *Application of a semi-implicit Euler method to mass action kinetics*, Chem. Eng. Sci. 36 (1981), no. 10, 1633–1641.
- [29] R. J. Renka. A simple explanation of the Sobolev gradient method. Unpublished, University of North Texas 2006.
- [30] B. Sportisse, *An analysis of operator splitting techniques in the stiff case*, J. Comput. Phys. 161 (2000), no. 1, 140–168.
- [31] J. Strikwerda, *Finite difference schemes and partial differential equations*, 2nd ed., SIAM, Philadelphia, PA, 2004.
- [32] W. T. Taitano, L. Chacon, and A. N. Simakov, *An adaptive, implicit, conservative, 1D-2V multi-species Vlasov–Fokker–Planck multi-scale solver in planar geometry*, J. Comput. Phys. 365 (2018), 173–205.
- [33] S. Tiwari, *Coupling of the Boltzmann and Euler equations with automatic domain decomposition*, J. Comput. Phys. 144 (1998), no. 2, 710–726.
- [34] J. S. Truelove, *Discrete-ordinate solutions of the radiation transport equation*, J. Heat Trans. ASME 109 (1987), no. 4.
- [35] J. Van Dijk, G. M. W. Kroesen, and A. Bogaerts, *Plasma modelling and numerical simulation*, J. Phys. D. Appl. Phys. 42 (2009), no. 19, 190301.
- [36] J. Vencels, G. L. Delzanno, A. Johnson, I. B. Peng, E. Laure, and S. Markidis, *Spectral solver for multi-scale plasma physics simulations with dynamically adaptive number of moments*, Proc. Comput. Sci. 51 (2015), 1148–1157.
- [37] A. A. Vlasov, *The oscillation properties of an electron gas*, US Atomic Energy Commission, Washington, DC, 1938.
- [38] Y. B. Zel'Dovich and Y. P. Raizer, *Physics of shock waves and high-temperature hydrodynamic phenomena*, Courier Corporation, North Chelmsford, Massachusetts, 2002.
- [39] X.-M. Zhu and P. Yi-Kang, *A simple collisional–radiative model for low-pressure argon discharges*, J. Phys. D. Appl. Phys. 40 (2007), no. 8, 2533.

How to cite this article: C. Lederman and D. Bilyeu, *Reducing the computational complexity of implicit schemes in the modeling of kinetic inelastic collisions in a partially ionized plasma*, Numer. Methods Partial Differ. Eq. (2024), e23121. <https://doi.org/10.1002/num.23121>

APPENDIX A: DETAILED EQUATIONS OF THE PHYSICAL MODEL

The analytic formulas for atomic loss and gain, respectively, due to electron impact excitation are given by:

$$g_{\text{exc loss}, N_n} = -N_n \int_v f(v) v \sigma_{\text{exc}}(v, n) dv, \quad (\text{A1})$$

$$g_{\text{exc gain}, N_m} = N_n \int_v f_{k_0} \delta_{k_0}(v) v \sigma_{\text{exc}}(v, n) dv. \quad (\text{A2})$$

From Equations (A1) and (A2), we derive Equations (8) and (10) by considering the contribution from the neighborhood of each k_0 , corresponding to electrons within a specified kinetic energy independently. The number of electrons gained and lost due to these collisional processes are also given by Equations (A1) and (A2). For de-excitation, atoms of state m and electrons with kinetic energy ε_1 are lost, while atoms of state n and kinetic energy ε_0 are gained ($\varepsilon_0 > \varepsilon_1$ and $n > m$ for both excitation and de-excitation with this notation). The atomic gain and loss terms are then given by:

$$g_{\text{de-exc loss}, N_m} = -N_m \int_{v'} f(v') v' \sigma_{\text{de-exc}}(v', n) dv', \quad (\text{A3})$$

$$g_{\text{de-exc gain}, N_n} = N_m \int_{v'} f(v') v' \sigma_{\text{de-exc}}(v', n) dv'. \quad (\text{A4})$$

For the numerical implementation, we use discretized versions analogous to Equations (8), (9), (10), and (11).

Atoms lost due to ionization are given by:

$$g_{\text{ion loss}, N_n} = -N_n \int_v \int_{v'} f(v) v \sigma_{\text{ion}}(v, v', n) dv dv'. \quad (\text{A5})$$

The number of ions gained is equal to the number of atoms lost:

$$g_{\text{ion gain}, N_+} = N_n \int_v \int_{v'} f(v) v \sigma_{\text{ion}}(v, v', n) dv dv'. \quad (\text{A6})$$

If we were only interested in changes in atomic and ion numbers, Equations (A5) and (A6) could be preintegrated in v' since the distribution f is only evaluated with respect to v . However, by holding the discretized versions of the right-hand side of Equations (A5) and (A6), given by the right-hand side of Equation (14), the location of one of the gained electrons can be determined.

$$\int_v g_{k_0} \delta_{k_0}(v) dv = -N_n \int_v \int_{v'} f_{k_0} \delta_{k_0}(v) v \delta_{k_1}(v') \sigma_{\text{ion}}(v, v', n) dv dv' \quad (\text{A7})$$

$$\int_v g_{k_1} \delta_{k_1}(v) dv = N_n \int_v \int_{v'} f_{k_0} \delta_{k_0}(v) v \delta_{k_1}(v') \sigma_{\text{ion}}(v, v', n) dv dv' \quad (\text{A8})$$

The location of the third electron, one of the two gained electrons, is established through conservation of energy as required by Equation (13). For recombination, the analytic ion loss and atom gain terms are given by:

$$g_{\text{rec loss}, N_+} = -N_+ \int_{v'} \int_{v''} f(v') v' f(v'') v'' \sigma_{\text{rec}}(v', v'', n) dv' dv'', \quad (\text{A9})$$

$$g_{\text{rec gain}, N_n} = N_+ \int_{v'} \int_{v''} f(v') v' f(v'') v'' \sigma_{\text{rec}}(v', v'', n) dv' dv''. \quad (\text{A10})$$

The discretized version of Equation (A9), given by Equation (15), is implemented by looping over k_1 and k_2 , with the energy of the gained electron being fixed by conservation of energy.

For a deeper discussion of the gain and loss formulas as a result of collisional processes and the exact cross-section formulas for a hydrogen atom, we refer to [25] and [38], respectively.

APPENDIX B: PATH CALCULATIONS

As explained in Section 4.3, the approximation of the sum of all possible paths between two states is performed by considering the direct paths between them and adding additional paths that begin and end at the same state. Direct paths can be computed recursively from paths of smaller lengths. These direct paths are allowed to have step sizes of 1, 2, or the entire path length, where paths of step size 1 make the largest contribution.

For paths that begin and end at the same state, we consider paths of step size 1. The smallest paths that begin and end at the same state have 2 steps, with two possibilities: $s \rightarrow s+1 \rightarrow s$ and $s \rightarrow s-1 \rightarrow s$. The 6 possible paths of length 4 are presented graphically in Figure B1. Four of the six paths are compilations of the length 2 paths. For paths of length 6 with all steps of length 1, there are

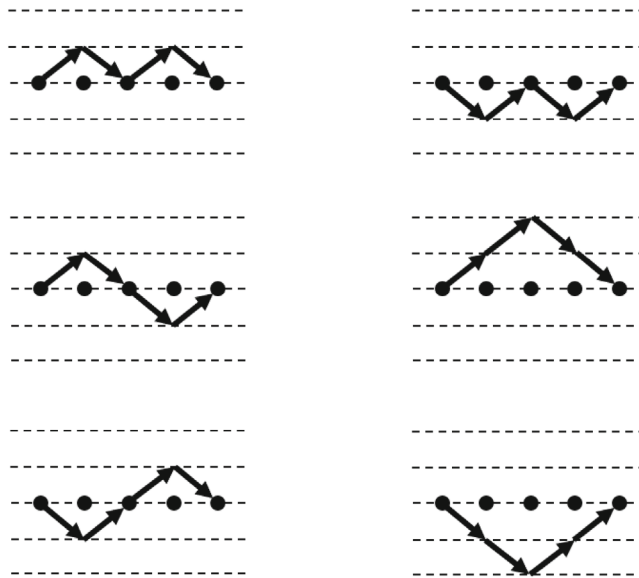


FIGURE B1 A pictorial example of the 6 possible paths that can be taken for 4 transitions such that the end state is the same as the starting state. For each of the 6 subplots, the dots going from left to right represent the number of steps taken, and the dashed lines going from bottom to top represent the change in state from the initial and range from -2 to 2 .

18 possible paths that start and stop at the same state. All but 2 of these are compositions of the length 2 and 4 paths. We computed paths of this type up to length 14 for each state s , where the contribution to the overall path value diminished for higher-length paths of this type.

APPENDIX C: OVERVIEW OF RELEVANT NUMERICAL METHODS

C.1 | Forward Euler

The Forward Euler method is given by:

$$u(t + dt) = u(t) + dt \cdot g(u(t)). \quad (C1)$$

While it is computationally cheaper than Backward Euler, the Forward Euler scheme is only conditionally stable and requires a very small time step dt for stability. It can provide accurate results, but it generates a different discrete solution compared to Backward Euler.

C.2 | Predictor-corrector methods

Predictor-Corrector methods [15] use a simpler scheme as a stepping stone toward calculating a more complex one. A simple fixed-point iteration approach to solving $G(u) = 0$, where G is defined by Equation (17), is given by:

$$u_{i+1} = u_i + G(u_i). \quad (C2)$$

Equation (C2) can also be seen as solving Equation (17) using Newton's method with the approximation that $u \gg g(u)$. However, this method may not offer stability benefits over explicit schemes and is not unconditionally stable.

C.3 | Implicit-explicit method

The Implicit-Explicit (IMEX) method is a relatively simple approach to achieving stability for linear rate equations at a lower computational cost. It involves advancing each variable implicitly with respect

to itself and explicitly with respect to the other variables [28]. The scheme is unconditionally stable only in the special case where $\frac{dg}{du}$ is diagonal, making it equivalent to Backward Euler as described in Section 3.1. However, in a more general case, where every variable of u is updated in proportion to its own size, as in the model under discussion, some stability improvement can be expected, although unconditional stability should not be assumed. The method can be presented as an iterative approach within a single time step:

$$D_{SIOE}(q, r)i = \begin{cases} dt \frac{dg}{du}[q, r]i - 1 & \text{if } q = r, \\ 0 & \text{if } q \neq r, \end{cases} \quad (C3)$$

$$u_{i+1} = u_i - (D_{SIOE})^{-1} G(u_i).$$

However, two major limitations of this approach should be considered. First, it fails to limit growth that arises from rapid decay at another location. For example, consider linear, but very rapid decay with a rate of $-\lambda_4$ (where $\lambda_4 \gg 0$) of the distribution function at a location f_{k_4} , with all of that lost mass transferred to f_{k_3} (where $g_{f_{k_3}} = \lambda_4 f_{k_4}$ and $g_{f_{k_4}} = -\lambda_4 f_{k_4}$). The decline in f_{k_4} will be $\frac{-dt\lambda_4}{1+dt\lambda_4} f_{k_4}$, ensuring that f_{k_4} will simply decline to 0. In contrast, f_{k_3} will increase explicitly by $dt\lambda_4 f_{k_4}$. In the next time step or iteration i , f_{k_4} will be small, and though f_{k_3} will stabilize, it will do so at an inaccurately high value, which adds mass to the overall system.

An additional problem arises with its application to nonlinear rate equations. The exclusive self-loss dependency, with other variable dependencies only contributing to growth, is a characteristic specific to linear rate equations. In contrast, for nonlinear decay processes, even a 1D cubic ordinary differential equation (ODE) with a positive variable decaying towards 0 can become negative with Backward Euler and a sufficiently large time step. The situation becomes even more complicated with other variables contributing to the decay in a nonlinear fashion. Consequently, the Implicit-Explicit method may not be a reliable choice for nonlinear systems.

C.4 | Sobolev regularization

Sobolev regularization is a brute force approach to achieve stability by modifying the scheme to reduce its measure in a Sobolev space [22–24, 29]. This method is theoretically well-founded, and the computational cost is relatively low ($O(L)$, where L is the number of variables). It involves applying iterative updates to a particular subset of variables within u , typically applied to smooth functions like the electron velocity distribution function (f). However, applying Sobolev regularization to the entire system may not be effective for all types of physical models.

An equation analogous to Equation (17), but only focusing on solving for the values of f , is given by:

$$f \equiv f(t + dt)G_f(u) = f + dt \cdot g_f(u) - f = 0. \quad (C4)$$

The regularization can be applied repeatedly in an iterative fashion as follows:

$$f_{i+1} = f_i - (I - K\Delta)^{-1}(-G_f(u_i)). \quad (C5)$$

In Equation (C5), Δ is the discrete Laplacian. The parameter K is generally omitted in theoretical studies of Sobolev spaces as the space is the same for any number $K > 0$. However, in the current numerical application, K needs to be carefully chosen to be large enough to damp out the rising instability while being small enough to avoid washing out the accuracy of the solution updates. An estimate for K can be obtained by comparing an arbitrary $g(u)$ to Δu .

To approach an estimate for K , it is noted that the Sobolev regularization exactly matches Newton's method for the specific case of Backward Euler applied to the heat equation:

$$f(t + dt) = f(t) + dt \cdot \alpha \triangle f(t + dt). \quad (C6)$$

$$K = dt \cdot \alpha.$$

It should be noted that Sobolev regularization is likely to perform better with diffusive processes, like the Fokker–Planck Equation, and may be less effective with other types of models (excitation, de-excitation, ionization, and recombination).

Sobolev regularization was implemented only for the electrons since it is typically applied to smooth functions like the electron VDF. The atomic numbers are indirectly affected in future iterations because they are highly dependent on f . One potential approach for the atomic numbers is to consider a discrete distribution of atomic numbers plotted against the excitation energy associated with each state. However, this approach presents challenges due to the vastly different rates of change between atomic numbers and the need to choose an appropriate regularization parameter K .

Overall, the choice of numerical method depends on the specific physical model being considered and the computational resources available. Some methods may perform better for certain types of models, while others may be more computationally efficient. Careful consideration and testing are required to determine the most suitable numerical approach for a given problem.

APPENDIX D: EXAMPLE SETUP DETAILS

D.1 | Example 1 setup

The exact formulas for the initial state of the electron distribution function, number of hydrogen atoms at each excitation level, and ion number for the first example are presented here. The equations include the following physical parameters: amagat (amg), the Rydberg unit of energy (Ry), the mass of an electron in electron volts (m_e), and Planck's constant (h). The temperature (T) is 2.5 eV and the number of allowable excitation levels (M) was set to 50. The number of atoms at each excitation level (N_m) is:

$$N_m = amg \frac{m^2 e^{-\left(1 - \frac{1}{m^2}\right)\left(\frac{Ry}{T}\right)}}{\sum_{m=1}^{m=M} m^2 e^{-\left(1 - \frac{1}{m^2}\right)\left(\frac{Ry}{T}\right)}}. \quad (D1)$$

The number of ions and electrons is:

$$N_e = N_+ = \sqrt{amg \left(\frac{2\pi m_e T}{h^2}\right)^{3/2}} e^{-\frac{Ry}{T}}. \quad (D2)$$

The electron VDF, with parameters $v_c = 10^5$ meters per second and $v_w = 5$, is:

$$f_g(v) = e^{-\frac{(v-v_c)^2 v_w}{v_c^2}},$$

$$f_w(v) = \left(\int f_g(v) dv\right) 2\pi \left(\frac{m_e}{2\pi T}\right)^{3/2} v^2 e^{-\frac{m_e v^2}{2T}}, \quad (D3)$$

$$f(v) = N_e \frac{f_g(v) + f_w(v)}{\int (f_g(v) + f_w(v)) dv}.$$

D.2 | Example 2 setup

For example 2, the atoms at each excitation level, electron number, and ion number are the same as example 1 and given by Equations (D1) and (D2). The electron VDF, with the parameter $\xi = 10^{-5}$, is:

$$f(v) = N_e \frac{e^{-\xi v}}{\int e^{-\xi v} dv}. \quad (D4)$$

D.3 | Example 3 setup

For this example, the atomic numbers are given by:

$$N_m = \begin{cases} amg & \text{if } m = 1, \\ 0 & \text{if } m > 1. \end{cases} \quad (\text{D5})$$

And with the parameter $\gamma = 10^{-8}$, the number of ions and electrons is:

$$N_e = N_+ = \gamma \cdot amg. \quad (\text{D6})$$

With parameters $v_c = 6 \times 10^6$ meters per second and $v_w = 400$, the electron VDF is:

$$f(v) = e^{-\frac{(v-v_c)^2 v_w}{v_c^2}}. \quad (\text{D7})$$